Part 4 - Pygame Car Crash Tutorial

Contents

Part 4 - Pygame Car Crash Tutorial.....1
config.py.....1
CarCrash Class 2
Player Class.....5
Explanation 6
Enemy Class.....7
Assignment Submission.....9

Time required: 30 minutes

Our game is still incomplete. There's no fun in playing a game with the same thing happening over and over again.

In this section we're going to move our classes into separate files, add movement, and cover Sprite Grouping.

config.py

Create a new file named **config.py**

Enter the following code.

```python
"""
    Filename: config.py
    Author:
    Date:
    Purpose: Global variables and constants for the entire program
"""
# import config module into all other modules

# Setup global constants and variables for screen size and speed
SURFACE_WIDTH = 400
SURFACE_HEIGHT = 600

# Global variable for speed across screen
speed = .3
```

The config file stores global constants and variables for all the program files. It must be imported into each of the program files.

CarCrash Class

The changes are in green.

```python
"""
    Filename: car_crash_4.py
    Author:
    Date:
    Purpose: Main logic for the program
"""

# Import modules
import pygame
import sys

# Import our game classes
import player
import enemy
import config
```

Remove the following lines. We will create the player and enemy as sprites later on in the program.

```
# Create a Player and Enemy object
self.player = player.Player()
self.enemy = enemy.Enemy()
```

Add the lines marked in green.

```
class CarCrash:
    def __init__(self):
        # Initialize the pygame library
        pygame.init()

        # Setup a computer clock object
        self.CLOCK = pygame.time.Clock()

        # Create the game window
        self.surface = pygame.display.set_mode(
            (config.SURFACE_WIDTH, config.SURFACE_HEIGHT)
        )

        # Load background image from file
        self.background = pygame.image.load("./content/street.png")

        # Set window caption
        pygame.display.set_caption("Car Crash")

        # Create the player and enemy sprites
        self.create_sprites()
```

We changed the location of the SURFACE_WIDTH and SURFACE_HEIGHT constants to the config file.

Update the run_game method to reflect the following code.

```python
# ----------------------- RUN GAME ----------------------------------------#
    def run_game(self):
        """Start the infinite Game Loop"""
        while True:
            # Change in delta time for each frame
            # Multiplying movement by dt allows all computers
            # to run the game at the same speed
            # 100 frames per second
            self.dt = self.CLOCK.tick(100)

            # Closing the program by clicking the X
            # causes the QUIT event to be fired
            for event in pygame.event.get():
                # Exit game if window is closed
                if event.type == pygame.QUIT:
                    # Quit Pygame
                    pygame.quit()
                    # Exit Python
                    sys.exit()

        # ------------------- DRAW ON BACKBUFFER ---------------------------#
            # Draw everything on the backbuffer first
            # Fill the surface with the background image loaded earlier
            self.surface.blit(self.background, (0, 0))

            # Move and Re-draw all Sprites
            for entity in self.all_sprites:
                self.surface.blit(entity.image, entity.rect)
                entity.move(self.dt)

            # Redraw the surface
            pygame.display.update()
```

Add the new create_sprites method to the end of the program.

```python
# ----------------------- CREATE SPRITES -----------------------------------#
    def create_sprites(self):
        # Create a Player and Enemy object
        player_sprite = player.Player()
        enemy_sprite = enemy.Enemy()

        # Create Sprites Groups, add Sprites to Groups
        # A separate enemies group is created,
        # to allow for more enemy Sprites later on if needed
        enemies = pygame.sprite.Group()
        enemies.add(enemy_sprite)

        # This group includes all Sprites
        self.all_sprites = pygame.sprite.Group()
        self.all_sprites.add(player_sprite)
        self.all_sprites.add(enemy_sprite)


# Call the main function
if __name__ == '__main__':
    # Create game instance
    car_crash = CarCrash()
    # Start the game
    car_crash.run_game()
```

Here we've created "groups" for our sprites. A Sprite group is sort of like a classification. It's much easier to deal with 2 or 3 groups, rather than having to deal with dozens or even hundreds of sprites. Keeping them in one group allows us to easily access every sprite in that group.

In the example above, we've created two groups one called **enemy** and the other called **all_sprites**. This code doesn't have more than one enemy, but since multiple enemies could easily be added here, we've created a separate group for it. To add a Sprite to a group, use the **add()** function.

Player Class

Add the lines in green to the player class.

```python
"""
    Filename: player.py
    Author:
    Date:
    Purpose: All logic for the player's car is in this class
"""

import pygame
# config.py contains global variables and constants
import config
```

Each file imports the config file.

Replace the draw method with the move method.

```python
# ---------------------------- MOVE ---------------------------------------#
    def move(self, dt):
        # Called each time through the Game Loop
        # Read the keyboard to see if any keys pressed
        pressedKeys = pygame.key.get_pressed()

        # Keep the player on the screen
        # The sprite can't move past the left edge of the surface
        if self.rect.left > 0:

            # Left arrow key pressed, move left 5 pixels at a time
            if pressedKeys[pygame.K_LEFT]:
                self.rect.move_ip(-5, 0)

        # The sprite can't move past the right edge of the surface
        if self.rect.right < config.SURFACE_WIDTH:

            # Right arrow key pressed, move right 5 pixels
            if pressedKeys[pygame.K_RIGHT]:
                self.rect.move_ip(5, 0)
```

Explanation

We define a **move** method for the Player class that controls the movement of the player. When this function is called, it checks to see if any keys are pressed down or not.

The **if** statements check for 2 keys, **LEFT** and **RIGHT**. If the **if** statement proves true, then the **move_ip()** method is called on **Player.rect** moving it in a certain direction. The

move_ip() takes two parameters, the first representing the distance to be moved in the X direction and second, the distance to be moved in the Y direction.

There are two if statements, as we are testing for two separate event possibilties.

The two if statements, **if self.rect.left > 0:** and **if self.rect.left < config.SCREEN_WIDTH:** ensure that the player isn't able to move off screen.

Enemy Class

Modify your enemy class as shown below.

```python
"""
    Name: enemy.py
    Author:
    Date:
    Purpose: All logic for the enemy's car is in this class
"""

import pygame
import random

# config.py contains global variables and constants
import config


class Enemy(pygame.sprite.Sprite):
    """Define the enemy class and methods"""
# ------------------------- INITIALIZE ENEMY OBJECT ------------------------#

    def __init__(self):

        # Construct an enemy object from Sprite class
        super().__init__()

        # Load enemy car image from file into a variable
        self.image = pygame.image.load("./content/enemy.png")

        # Create a surface rectangle the same size as the enemy car image
        self.surface = pygame.Surface((50, 80))

        # Create a rectangle with a random X location
        # Stay 40 pixels from the left and right edge
        self.rect = self.surface.get_rect(
            center=(random.randint(40, config.SURFACE_WIDTH - 40),  0)
        )
```

Remove the draw method.

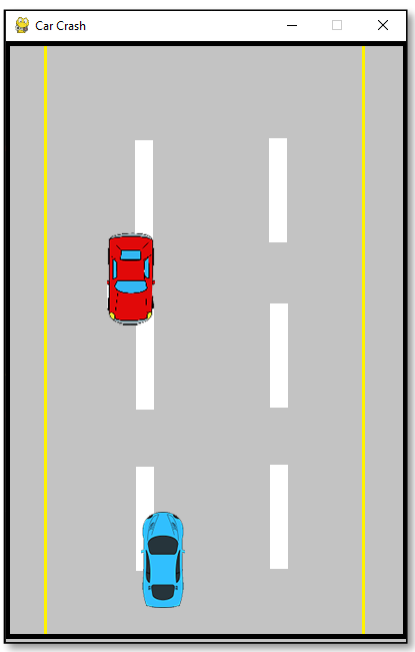
Add the following move method.

```python
    # ------------------------- MOVE-------------------------------------#
    def move(self, dt):
        # Move the sprite down SPEED pixels at a time
        self.rect.move_ip(0, config.speed * dt)

        # When the sprite reaches the botton of the surface,
        # Move to the top, random center location on the X axis
        if (self.rect.bottom > config.SURFACE_HEIGHT):
            self.rect.top = 0
            self.rect.center = (random.randint(30, 370), 0)
```

The enemy class also has a **move** method. It first calls the **move_ip()** function, moving the Enemy object down by **speed** pixels at a time. Next it checks to see if the top of the Enemy has reached the end of the screen. If True, it resets it back to the top of screen at a random location on the X axis.

Example run:



The player car moves back and forth with the left and right cursor keys.

The enemy car goes down the road nice and smooth at a random location on the X axis.

Assignment Submission

Zip up the program files folder and submit in Blackboard.